

10 System Design Solutions

End-to-End Designs for Real-World Applications — Interview Reference

Each design follows the interview framework: Requirements → Capacity Estimation → Architecture → Key Components → Trade-offs

Design 1: Video Streaming Platform

Like Netflix — upload, transcode, stream video to millions of concurrent users

Requirements

Functional	Non-Functional
Upload video (creators) Stream video (viewers) Search / browse content Resume playback Recommendations	500M+ users, 100M concurrent streams Video: start < 2 sec, no buffering 99.99% availability Global CDN delivery Support 4K, 1080p, 480p adaptive bitrate

Capacity Estimation

Users: 500M. Daily active: 100M. Avg watch: 2 hr/day Bandwidth per stream: 5 Mbps (1080p) Peak concurrent: 50M streams $\times$ 5 Mbps = 250 Tbps egress Storage: 10k new videos/day $\times$ 10 GB avg = 100 TB/day raw After transcoding (5 formats $\times$ 3 resolutions): ~3 PB/day stored

Architecture

```
+-----+ Creator -----| Upload |----[ Transcoding Queue (Kafka) ] | Service | | +-----+ [
Transcoding Workers ] (FFmpeg, multiple resolutions) | [ S3/Object Store ] | [ CDN (CloudFront/Akamai) ] |
Viewer ----[ API Gateway ]----[ Video Service ]----[ CDN PoP ] (Auth, Rate Limit) (metadata DB) (edge cache) |
[ Recommendation Service ] (ML model, Spark, Redis cache)
```

Key Components

Component	Technology / Design Decision
Upload	Chunked multipart upload to S3. Resume support via S3 multipart upload ID. Max chunk: 100MB.
Transcoding	Kafka queue triggers transcoding workers. Parallel FFmpeg jobs: 4K/1080p/720p/480p/360p + HLS/DASH segments. Store in S3 with manifest file.
Streaming protocol	HLS (HTTP Live Streaming): .m3u8 playlist + .ts segments. Adaptive bitrate: client switches quality based on bandwidth. DASH for non-Apple.
CDN strategy	Videos stored at S3 origin. CDN caches popular segments at edge PoPs worldwide. Cache-Control: max-age=31536000. Long-tail content stays at origin.
Metadata DB	Cassandra for video metadata (id, title, description, thumbnails, formats available, duration). Wide-column scales to billions of videos.
Resume playback	Store last_position per userId+videoid in Redis (fast writes during playback). Sync to Cassandra every 30s and on pause/exit.
Search	Elasticsearch indexes title, description, tags. Auto-suggest via prefix search. Weighted by views, recency, user preferences.
Recommendations	Offline: collaborative filtering (Spark ML, matrix factorization). Online: user click/watch history in Kafka -> real-time feature store (Redis) -> served by recommendation API.

Design 2: Ride-Sharing Platform

Like Uber — real-time location matching between riders and drivers

Requirements

Functional	Non-Functional
Rider: request ride, track driver, pay Driver: go online, accept trips, navigate Real-time location tracking ETA calculation Surge pricing	10M+ rides/day, global Location update: every 4 seconds per driver Driver matching: < 5 sec 99.99% availability Strong consistency for payments

Architecture

Driver App ---[WebSocket / gRPC]--- Location Service ---[Redis Geo]--- Matching Service | | [Kafka stream] [Supply Service] | (driver availability) [Trip Service] -----[PostgreSQL] (trip records) | [Pricing Service] (surge) | [Payment Service] (Stripe)

Key Components

Component	Design
Location tracking	Drivers send GPS every 4s via WebSocket. Location Service writes to Redis Geo (GEOADD). Kafka publishes stream for downstream consumers (analytics, ETL).
Driver matching	Rider requests ride with pickup location. Matching Service does GEORADIUS in Redis to find nearby available drivers (r=2km). Filter by driver rating, vehicle type. Match best candidate. Fallback: expand radius.
Geospatial indexing	Redis GEO commands: GEOADD (store), GEORADIUS (find nearby), GEODIST (distance). Internally uses sorted set with geohash scores. O(N+log M) lookup.
Trip state machine	States: REQUESTED -> DRIVER_ASSIGNED -> DRIVER_ARRIVING -> IN_PROGRESS -> COMPLETED. Stored in Redis (fast) + PostgreSQL (durable). State changes published to Kafka.
Surge pricing	Demand/supply ratio per geohash cell. Computed every 60s from driver density (Redis Geo count) and pending requests (in-memory counter). Surge multiplier stored in Redis with TTL.
ETA calculation	Road graph stored in specialized routing engine (OSRM or Google Maps API). Pre-computed routes cached in Redis. Real-time traffic adjustments via Kafka stream of GPS data.
Payment	Stripe for card processing. Idempotency key = tripId to prevent double charges. Two-phase: pre-auth on trip start, capture on completion. Wallet balance in PostgreSQL with optimistic locking.

Design 3: Search Autocomplete / Typeahead

Like Google search suggestions — real-time prefix completion

Requirements

Return top-5 suggestions for prefix as user types. <100ms latency. Personalized suggestions. Handle billions of queries/day.

Architecture

User types "java" -> Browser ----[API Gateway] ----[Autocomplete Service] | +-----+-----+ | | [Redis] [Trie] [Postgres] (hot top-K) (prefix) (full dataset)

Component	Design
Trie (prefix tree)	Each node = character. Path from root to node = prefix. Each node stores top-K suggestions (sorted by frequency). Built offline from query logs. Stored in Redis or specialized Trie service (serialized).
Frequency ranking	Query logs processed by Spark daily. Frequency score = count weighted by recency (exponential decay). Top 5 per prefix stored in each Trie node. Rebuild nightly.
Hot prefix cache	Redis stores: "prefix:java" -> ["java tutorial","java interview","java spring boot",...]. TTL 1 hour. Cache invalidated on Trie rebuild. 99% of queries hit Redis.
Personalization	After showing top global suggestions, append user's recent searches from Redis (userId key, sorted set of recent queries). Blend: 3 global + 2 personal.
Update pipeline	Kafka stream: query events. Flink counts per prefix in 1-min windows. Updates Redis sorted set (ZADD, ZREVRANGE). Near-real-time ranking without full rebuild.
Scaling	Partition Trie by first letter (A-Z = 26 shards). Route requests by first char of prefix. Each shard = ~1/26 of dataset. Consistent hashing for distribution.

Design 4: Payment Gateway

Like Stripe — accept, process, and settle card payments with idempotency and strong consistency

Architecture

Merchant ----[Payment API] ----[Idempotency Layer] ----[Payment Processor] | (Redis key-value)
(Visa/Mastercard network) [Audit Log] | (Kafka, S3) [Settlement Service] | (batch, T+1 or T+2) [Ledger DB]
[Notification Service] (PostgreSQL, double-entry) (Kafka -> email/webhook)

Component	Design
Idempotency	Every payment request carries idempotency key (UUID from merchant). Redis stores key -> response for 24h. If key seen: return cached response without reprocessing. Prevents double charges on retry.
State machine	INITIATED -> AUTHORIZED -> CAPTURED -> SETTLED / FAILED / REFUNDED. Each transition stored in Postgres with optimistic locking (version column). Kafka event per transition.
Ledger (double-entry)	Every money movement = two ledger entries (debit + credit) that sum to zero. Append-only table. Balance = SUM of entries for account. Immutable history. Standard accounting model.
Fraud detection	Rule engine (velocity checks, geolocation mismatch, device fingerprint) runs synchronously in <100ms. ML model runs async; can trigger review or block. Thresholds per merchant tier.
Strong consistency	Payments use single-region PostgreSQL with SERIALIZABLE isolation for money movements. No eventual consistency for financial data. Distributed: use saga pattern with compensating transactions.
PCI DSS compliance	Card data: tokenize immediately (never store raw PAN). Use Vault or payment network tokenization. All transmission TLS 1.2+. Audit log immutable (Kafka + S3 Glacier). Access controls strict RBAC.

Design 5: Online Booking System

Like Booking.com — hotel/flight booking with inventory and concurrency control

Key Challenge: Preventing Double Booking

```
-- Optimistic locking approach: UPDATE inventory SET available_count = available_count - 1, version = version + 1 WHERE room_id = :roomId AND check_in = :date AND available_count > 0 AND version = :expectedVersion; -- If 0 rows updated: conflict -> retry or fail -- Or: Pessimistic locking (SELECT FOR UPDATE) BEGIN; SELECT available_count FROM inventory WHERE room_id=:id AND date=:date FOR UPDATE; -- row-level lock IF available_count > 0: UPDATE ... SET available_count = available_count - 1; INSERT INTO bookings ...; COMMIT;
```

Component	Design
Search	Elasticsearch for property search (location, amenities, price range). Results include real-time availability flag from inventory cache (Redis). Full re-check on booking.
Inventory	PostgreSQL table: (property_id, room_type, date, available_count). Partitioned by date range. Indexed on (property_id, date). Row-level locking for booking atomicity.
Reservation flow	1. User selects room. 2. Soft lock (Redis TTL 10 min): hold inventory temporarily. 3. User completes payment. 4. Hard lock: UPDATE inventory, create booking. 5. Release Redis lock. If payment fails: release soft lock, inventory available again.
Cancellation	Update booking status. Increment inventory count. Trigger refund workflow (Kafka event to payment service). Notification to user and property.
Pricing engine	Base price + demand multiplier (occupancy rate) + seasonal rules. Stored per date in pricing table. Updated by batch job or real-time pricing service. Cached in Redis for search display.
Overbooking prevention	Distributed lock via Redis SETNX for high-contention rooms. Or: queue all booking attempts for same room/date through a single-threaded booking worker (partition by roomId in Kafka).

Design 6: Social Media Analytics Dashboard

Real-time and batch analytics for post engagement, follower growth, reach

Architecture — Lambda Architecture

Events (likes, shares, views) ---[Kafka] ---+--- [Flink] --- [Redis] --- Dashboard | (real-time) (last 1h counters) | +--- [S3 / Parquet] --- [Spark] --- [Redshift/ClickHouse] (raw event store) (batch) (historical queries)

Component	Design
Event ingestion	Every like/comment/share/view = Kafka event. Schema Registry enforces Avro/Protobuf schema. Partition by postId for ordered processing per post.
Real-time (Flink)	Tumbling 1-min windows count events per post. Results stored in Redis counters (HINCRBY). Dashboard polls Redis for last-hour metrics. Sub-second latency.
Batch (Spark)	Daily Spark job reads raw Parquet from S3. Computes aggregate metrics: daily reach, follower growth, engagement rate. Writes to Redshift for BI tools (Tableau, Grafana).
Storage	Hot data (7 days): Redis counters. Warm data (90 days): ClickHouse (columnar, fast OLAP queries). Cold data (years): S3 Parquet, queried via Athena.
Pre-aggregation	For high-cardinality metrics (per-post per-hour), pre-aggregate in Flink and store aggregated rows. Querying 100 pre-aggregated rows beats scanning 10M raw events.
Dashboard API	GraphQL API. Query resolver: if time range < 1h: Redis. If < 90 days: ClickHouse. If older: Athena on S3. Transparent to frontend.

Design 7: E-Commerce Platform

Like Amazon — catalog, cart, orders, inventory, and recommendations at scale

Microservices Architecture

Client ---[API Gateway] ---+--- Product Service --- [Elasticsearch] (search) (Auth, Rate Limit) | --- [MongoDB] (catalog) +--- Cart Service --- [Redis] (session cart) +--- Inventory Service --- [PostgreSQL] (stock count) +--- Order Service --- [PostgreSQL] (orders) +--- Payment Service --- [Stripe API] +--- Notification Service --- [Kafka / SES] +--- Recommendation Service --- [ML model + Redis]

Component	Design
Product catalog	MongoDB: flexible schema for varied product attributes. Denormalized for read performance. Elasticsearch for full-text search, faceted filters (category, price, rating). Sync via CDC (Debezium) Mongo -> ES.
Shopping cart	Redis hash per userId: {productId: quantity}. TTL 7 days. Guest cart: cookie-based UUID. On login: merge guest and user carts (union with user preference priority). Fast reads/writes.
Inventory	PostgreSQL with optimistic locking (version column) or SELECT FOR UPDATE. Reserve inventory on order placed; release on cancellation. Async restock alerts via Kafka.
Order flow	Saga pattern: Order Service orchestrates. 1) Reserve inventory. 2) Process payment. 3) Confirm order. Each step has compensating transaction. Order state machine persisted in PostgreSQL.
Flash sale	Pre-compute stock in Redis. Use Lua script for atomic decrement (check-and-decrement). Queue overflow requests in Kafka. Consumer processes at sustainable rate. Prevents DB overload.
Recommendations	Item-based collaborative filtering (Spark ML). "Customers also bought" from co-purchase graph. Results cached in Redis by productId. Refreshed daily. Real-time: recent views from session data.

Design 8: Distributed File Storage

Like Dropbox/Google Drive — sync files across devices, share, collaborate

Architecture

Client ---[Sync Client]---[Block Server]--- S3 / GCS (file blocks) | | [Metadata DB] [CDN (for downloads)] (PostgreSQL: files, folders, versions, sharing permissions) | [Notification Service]---[WebSocket / SSE] | (push sync events) [Message Queue (Kafka)]---[Sync Worker]

Component	Design
Block-based storage	Files split into 4MB blocks. Each block: SHA256 hash = block ID. Only modified blocks are uploaded (delta sync). Blocks deduplicated: same content across users shares one block in object store. Client compares local SHA256 with last-sync SHA256 to detect changes.

Component	Design
Metadata DB	PostgreSQL tables: files (id, user_id, name, parent_folder_id, current_version_id), file_versions (id, file_id, block_ids[], created_at, size), blocks (id, sha256, storage_url, size). Hierarchical folder = adjacency list or nested set.
Conflict resolution	Last-write-wins with version vector. If two clients modify same file offline, create conflict copy ("file_conflict_2024.txt"). User manually resolves. Google Docs uses OT (Operational Transformation) for real-time collaborative editing.
Sync protocol	Client connects via WebSocket to notification service. On file change: server sends {fileId, version}. Client fetches new blocks delta. Offline: queue changes locally, sync when reconnected. Ordering via logical timestamp.
Sharing	Shared links: signed URL with expiry (S3 pre-signed URLs). Folder sharing: ACL table (resource_id, user_id, permission: VIEW/EDIT/ADMIN). Recursive permission inheritance via closure table or CTE.
Chunked upload	Large files: chunk and upload in parallel. Server assembles blocks. S3 multipart upload. Client tracks uploaded chunks (in Redis, per upload session). Resume on connection drop.

Design 9: Online Banking System

Core banking — accounts, transfers, transactions, fraud detection

Architecture

Customer Portal / App | [API Gateway] --- [Auth Service (OAuth2 + MFA)] | +--- [Account Service] --- [PostgreSQL, SERIALIZABLE isolation] +--- [Transfer Service] --- [Saga Orchestrator] --- [Account Service] +--- [Transaction Service] --- [TimescaleDB / PostgreSQL] +--- [Fraud Service] --- [Rule Engine + ML Model] +--- [Notification Service] --- [Kafka + Email/SMS]

Component	Design
Account balance	PostgreSQL with double-entry ledger. Balance = SUM(credit) - SUM(debit) — never stored directly, derived from ledger. Prevents balance discrepancy from concurrent updates. All rows append-only.
Transfer (saga)	Debit source -> Credit destination. If credit fails: compensating debit on destination + credit on source. Idempotency key = transferId (UUID). Saga log persisted so incomplete sagas can be retried or rolled back.
Isolation	SERIALIZABLE isolation for all money movements. Prevents phantoms and concurrent anomalies. Accepted trade-off: lower throughput than READ COMMITTED. Partition by account shard for scale.
Fraud detection	Velocity rules (5+ transactions in 60s), geolocation (different country in 30 min), anomaly ML model (Isolation Forest on spending patterns). Block or step-up auth (2FA challenge). Async for most; sync block for high-risk.
Regulatory compliance	Audit log: every action, actor, timestamp. Immutable (append-only table + S3 archival). 7-year retention. GDPR: right to erasure = pseudonymize (replace PII with token, delete mapping). PCI DSS for card data.
High availability	Active-active PostgreSQL across 2 regions with synchronous replication for consistency. Automatic failover via Patroni. RPO=0, RTO<30s. All writes go to primary; reads can go to replica for non-critical queries.

Design 10: Multi-Channel Notification System

Push, email, SMS, in-app — at scale with reliability and user preferences

Architecture

Event Source (Order, Alert...) ---[Kafka] ---[Notification Service] | +-----+-----+-----+ | | [Push] [Email] [SMS] [In-App] (FCM/APNs)(SES/S6) (Twilio) (WebSocket) | | [Retry] ---[DLQ] [Audit Log] (Exponential (Kafka + S3) backoff)

Component	Design
User preferences	MongoDB: {userId, channels: {push: true, email: false, sms: true}, quiet_hours: "22:00-07:00", frequency_cap: "max 5/day per category"}. Read on every notification send to respect preferences. Cache in Redis (5 min TTL).

Component	Design
Template engine	Thymeleaf/Freemarker for email HTML. Templates stored in DB with version history. Variables injected: {userName}, {orderNumber}, etc. Multi-language: locale-aware template selection. A/B test different templates via feature flag.
Priority queuing	Kafka topics by priority: notifications.critical (fraud alerts, OTP), notifications.transactional (order updates), notifications.marketing (promotions). Consumers process critical first. Marketing rate-limited.
Rate limiting	Per-user frequency cap enforced via Redis sliding window counter. Key: "notif:{userId}:{category}:{window}". If count > limit: drop or schedule for next window. Never spam users.
Retry / reliability	Failed sends: exponential backoff retry (1s, 2s, 4s, 8s max). After N retries: dead letter queue. Alert ops team. Idempotency: each notification has ID; provider deduplication via idempotency key (Sendgrid, Twilio both support this).
Delivery tracking	Webhook from providers: delivered, opened, bounced, failed. Kafka event per status update. Stored in time-series DB (TimescaleDB). Dashboard shows delivery rates, open rates, bounce rates per template.
Fan-out at scale	1 event -> 10M users (marketing blast). Pre-compute recipient list offline (Spark job). Write notification records to DB in batches of 1000. Kafka workers pick up batches and send. Avoid generating 10M Kafka messages at once.

System Design Cheat Sheet — Decision Tables

Storage Selection

Data Type	Best Storage	Reason
User profiles, docs	PostgreSQL / MongoDB	Relational integrity or flexible schema
Session / cache	Redis	Sub-ms reads, TTL, atomic ops
Time-series metrics	TimescaleDB / InfluxDB	Efficient time-range queries, compression
Full-text search	Elasticsearch	Inverted index, scoring, facets
File / binary / video	S3 / GCS / Azure Blob	Cheap, durable, CDN-ready
Analytics / OLAP	ClickHouse / Redshift / BigQuery	Columnar, fast aggregation
Graph relationships	Neo4j / Amazon Neptune	Efficient graph traversals
Geospatial	Redis Geo / PostGIS	GEORADIUS, spatial indexes